

Class 2 HW - Student

```
# fit_data: fit a workflow to a full dataset using a formula and parsnip spec
fit_data <- function(formula, model, data, ...) {
  wf <- workflows::workflow() |>
    workflows::add_formula(formula) |>
    workflows::add_model(model)
  fit(wf, data, ...)
}

# fit_split: fit a workflow and evaluate on a held-out split (last_fit pattern)
fit_split <- function(formula, model, split, ...) {
  wf <- workflows::workflow() |>
    workflows::add_model(model) |>
    workflows::add_formula(
      formula,
      blueprint = hardhat::default_formula_blueprint(
        indicators = FALSE, allow_novel_levels = TRUE
      )
    )
  tune::last_fit(wf, split, ...)
}
```

1

Create a parsnip specification for a linear regression model.

```
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.5.3
```

```
## -- Attaching packages ----- tidymodels 1.5.0 --
```

```
## v broom      1.0.12    v recipes      1.3.3
## v dials      1.4.3      v rsample      1.3.2
## v dplyr      1.2.0      v tailor       0.1.0
## v ggplot2    4.0.3      v tidyr        1.3.2
## v infer      1.1.0      v tune         2.1.0
## v modeldata  1.5.1      v workflows    1.3.0
## v parsnip    1.6.0      v workflowsets 1.1.1
## v purrr      1.2.1      v yardstick    1.4.0
```

```
## Warning: package 'dials' was built under R version 4.5.3
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## Warning: package 'infer' was built under R version 4.5.3

## Warning: package 'modeldata' was built under R version 4.5.3

## Warning: package 'parsnip' was built under R version 4.5.3

## Warning: package 'recipes' was built under R version 4.5.3

## Warning: package 'rsample' was built under R version 4.5.3

## Warning: package 'tailor' was built under R version 4.5.3

## Warning: package 'tune' was built under R version 4.5.3

## Warning: package 'workflows' was built under R version 4.5.3

## Warning: package 'workflowsets' was built under R version 4.5.3

## Warning: package 'yardstick' was built under R version 4.5.3

## -- Conflicts ----- tidymodels_conflicts() --
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step() masks stats::step()
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'

## The following object is masked from 'package:dplyr':
##
##      select
```

```
library(ISLR)
```

```
## Warning: package 'ISLR' was built under R version 4.5.3
```

HW code

```
lm_spec <- linear_reg() %>%
  set_mode("regression") %>%
  set_engine("lm")
```

2

Once we have the specification we can `fit` it by supplying a formula expression and the data we want to fit the model on. The formula is written on the form `y ~ x` where `y` is the name of the response and `x` is the name of the predictors. The names used in the formula should match the names of the variables in the data set passed to `data`.

Use `lm_spec` to fit the model of `medv` predicted by every predictor variable. Hint: you can use `“.”` to specify every predictor.

HW code

```
lm_fit <- lm_spec %>%  
  fit(medv ~ ., data = Boston)  
lm_fit  
  
## parsnip model object  
##  
##  
## Call:  
## stats::lm(formula = medv ~ ., data = data)  
##  
## Coefficients:  
## (Intercept)      crim          zn          indus          chas          nox  
##  3.646e+01  -1.080e-01   4.642e-02   2.056e-02   2.687e+00  -1.777e+01  
##          rm          age          dis          rad          tax          ptratio  
##  3.810e+00   6.922e-04  -1.476e+00   3.060e-01  -1.233e-02  -9.527e-01  
##      black      lstat  
##  9.312e-03  -5.248e-01
```

3

Get a summary of your model using `pluck` and `summary`

HW code

```
lm_fit %>%  
  pluck("fit") %>%  
  summary()  
  
##  
## Call:  
## stats::lm(formula = medv ~ ., data = data)  
##  
## Residuals:  
##      Min       1Q   Median       3Q      Max
```

```
## -15.595 -2.730 -0.518 1.777 26.199
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.646e+01  5.103e+00  7.144 3.28e-12 ***
## crim        -1.080e-01  3.286e-02 -3.287 0.001087 **
## zn           4.642e-02  1.373e-02  3.382 0.000778 ***
## indus        2.056e-02  6.150e-02  0.334 0.738288
## chas         2.687e+00  8.616e-01  3.118 0.001925 **
## nox          -1.777e+01  3.820e+00 -4.651 4.25e-06 ***
## rm           3.810e+00  4.179e-01  9.116 < 2e-16 ***
## age          6.922e-04  1.321e-02  0.052 0.958229
## dis          -1.476e+00  1.995e-01 -7.398 6.01e-13 ***
## rad           3.060e-01  6.635e-02  4.613 5.07e-06 ***
## tax          -1.233e-02  3.760e-03 -3.280 0.001112 **
## ptratio      -9.527e-01  1.308e-01 -7.283 1.31e-12 ***
## black         9.312e-03  2.686e-03  3.467 0.000573 ***
## lstat        -5.248e-01  5.072e-02 -10.347 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.745 on 492 degrees of freedom
## Multiple R-squared:  0.7406, Adjusted R-squared:  0.7338
## F-statistic: 108.1 on 13 and 492 DF, p-value: < 2.2e-16
```

4

Take a look at `lm_fit` with `tidy`

HW Code

```
tidy(lm_fit)
```

```
## # A tibble: 14 x 5
##   term          estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)    36.5         5.10         7.14 3.28e-12
## 2 crim          -0.108        0.0329      -3.29 1.09e- 3
## 3 zn             0.0464        0.0137       3.38 7.78e- 4
## 4 indus          0.0206        0.0615       0.334 7.38e- 1
## 5 chas           2.69         0.862        3.12 1.93e- 3
## 6 nox           -17.8         3.82        -4.65 4.25e- 6
## 7 rm             3.81         0.418        9.12 1.98e-18
## 8 age            0.000692      0.0132       0.0524 9.58e- 1
## 9 dis           -1.48         0.199       -7.40 6.01e-13
## 10 rad           0.306        0.0663       4.61 5.07e- 6
## 11 tax          -0.0123      0.00376      -3.28 1.11e- 3
## 12 ptratio      -0.953        0.131       -7.28 1.31e-12
## 13 black         0.00931      0.00269       3.47 5.73e- 4
## 14 lstat        -0.525        0.0507     -10.3 7.78e-23
```

5

Extract the model statistics using `glance`

#HW code

```
glance(lm_fit)
```

```
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##   <dbl>      <dbl> <dbl>      <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     0.741      0.734  4.75      108. 6.72e-135    13 -1499. 3028. 3091.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

6

Get the predicted `medv` values from your model using `predict`

#HW code

```
predict(lm_fit, new_data = Boston)
```

```
## # A tibble: 506 x 1
##   .pred
##   <dbl>
## 1  30.0
## 2  25.0
## 3  30.6
## 4  28.6
## 5  27.9
## 6  25.3
## 7  23.0
## 8  19.5
## 9  11.5
## 10 18.9
## # i 496 more rows
```

7

Bind the predicted columns to your existing data

#HW code

```
bind_cols(
  predict(lm_fit, new_data = Boston),
  Boston
) %>%
  dplyr::select(medv, .pred)
```

```
## # A tibble: 506 x 2
##   medv .pred
##   <dbl> <dbl>
## 1 24    30.0
## 2 21.6  25.0
## 3 34.7  30.6
## 4 33.4  28.6
## 5 36.2  27.9
## 6 28.7  25.3
## 7 22.9  23.0
## 8 27.1  19.5
## 9 16.5  11.5
## 10 18.9  18.9
## # i 496 more rows
```

8

Now, make things easier by just using the `augment` function to do this.

#HW code

```
augment(lm_fit, new_data = Boston)
```

```
## # A tibble: 506 x 16
##   .pred .resid   crim   zn indus  chas  nox   rm  age  dis  rad  tax
##   <dbl> <dbl>   <dbl> <dbl> <dbl> <int> <dbl> <dbl> <dbl> <dbl> <int> <dbl>
## 1 30.0 -6.00  0.00632  18   2.31    0 0.538  6.58  65.2  4.09    1  296
## 2 25.0 -3.43  0.0273   0   7.07    0 0.469  6.42  78.9  4.97    2  242
## 3 30.6  4.13  0.0273   0   7.07    0 0.469  7.18  61.1  4.97    2  242
## 4 28.6  4.79  0.0324   0   2.18    0 0.458  7.00  45.8  6.06    3  222
## 5 27.9  8.26  0.0690   0   2.18    0 0.458  7.15  54.2  6.06    3  222
## 6 25.3  3.44  0.0298   0   2.18    0 0.458  6.43  58.7  6.06    3  222
## 7 23.0 -0.102 0.0883  12.5  7.87    0 0.524  6.01  66.6  5.56    5  311
## 8 19.5  7.56  0.145   12.5  7.87    0 0.524  6.17  96.1  5.95    5  311
## 9 11.5  4.98  0.211   12.5  7.87    0 0.524  5.63  100   6.08    5  311
## 10 18.9 -0.0203 0.170   12.5  7.87    0 0.524  6.00  85.9  6.59    5  311
## # i 496 more rows
## # i 4 more variables: ptratio <dbl>, black <dbl>, lstat <dbl>, medv <dbl>
```

9

Focus specifically on the median value and the `.pred`, then you can select those two columns

#HW code

```
augment(lm_fit, new_data = Boston) %>%
  dplyr::select(medv, .pred)
```

```
## # A tibble: 506 x 2
##   medv .pred
##   <dbl> <dbl>
```

```
## 1 24 30.0
## 2 21.6 25.0
## 3 34.7 30.6
## 4 33.4 28.6
## 5 36.2 27.9
## 6 28.7 25.3
## 7 22.9 23.0
## 8 27.1 19.5
## 9 16.5 11.5
## 10 18.9 18.9
## # i 496 more rows
```

10

Create a recipe with an interaction step between lstat and age

#HW code

```
rec_spec <- recipe(medv ~ ., data = Boston) %>%
  step_interact(~ lstat:age)
```

11

Create a workflow and add your lm_spec model and your rec_spec recipe.

#HW code

```
lm_wf <- workflow() %>%
  add_model(lm_spec) %>%
  add_recipe(rec_spec)
```

12

Fit your workflow.

#HW code

```
lm_wf %>% fit(Boston)
```

```
## == Workflow [trained] =====
## Preprocessor: Recipe
## Model: linear_reg()
##
## -- Preprocessor -----
## 1 Recipe Step
##
## * step_interact()
##
## -- Model -----
```

```
##
## Call:
## stats::lm(formula = ..y ~ ., data = data)
##
## Coefficients:
## (Intercept)      crim          zn          indus          chas          nox
##  38.034385   -0.111723    0.043109    0.018945    2.709592   -18.026578
##          rm          age          dis          rad          tax          ptratio
##   3.759388   -0.010315   -1.475346    0.307676   -0.012335   -0.958239
##       black       lstat  lstat_x_age
##   0.009187   -0.635696    0.001257
```